

# PLAN DE PRUEBAS

## STP - HU\_03 — Recepción de mensajes por el suscriptor

Sistema de mensajería MQTT · Axon Tech

|                     |                                                 |               |                  |
|---------------------|-------------------------------------------------|---------------|------------------|
| ID del Plan         | STP-HU03-2026                                   | Versión       | 1.0              |
| Proyecto            | Sistema MQTT – Axon Tech                        | Fecha         | 06/05/2026       |
| Historia de Usuario | HU_03 – Recepción de mensajes por el suscriptor | Estado        | En revisión      |
| Caso de Uso         | GU.HU3.UC1                                      | Preparado por | Nicolas Saavedra |
| Estándar principal  | ISO/IEC/IEEE 29119-3:2021                       | Aprobado por  | —                |
| Estándares de apoyo | ISO/IEC 12207:2017 · ISO/IEC 25010:2023         |               |                  |

# 1. Introducción

## 1.1 Propósito

Este documento constituye el **Plan de Pruebas del Software (STP)** para la historia de usuario HU\_03 del proyecto Axon Tech, elaborado conforme a la norma **ISO/IEC/IEEE 29119-3:2021** (Software and systems engineering – Software testing – Part 3: Test documentation). Su objetivo es especificar el alcance, el enfoque, los recursos, la planificación y los criterios de éxito de las actividades de prueba asociadas al caso de uso GU.HU3.UC1.

## 1.2 Marco normativo

El plan se articula sobre tres normas complementarias:

- ISO/IEC/IEEE 29119-3:2021 — Proporciona la estructura y los artefactos del plan de pruebas: niveles, tipos, técnicas de diseño, criterios de entrada/salida y métricas.
- ISO/IEC 12207:2017 — Define los procesos del ciclo de vida del software y justifica la distinción entre pruebas unitarias (proceso 6.4.3), de integración (proceso 6.4.4) y de sistema (proceso 6.4.5).
- ISO/IEC 25010:2023 — Establece el modelo de calidad del producto de software y permite mapear cada caso de prueba a las características de calidad que valida: Fiabilidad, Seguridad, Compatibilidad, Rendimiento y Mantenibilidad.

## 1.3 Alcance

Las pruebas cubren la implementación del componente suscriptor MQTT (**src/subscriber.py**) y su módulo de validación de mensajes (**src/validador.py**) para el flujo básico, los flujos alternos y el flujo de excepción del UC GU.HU3.UC1.

Quedan fuera del alcance de este plan:

- El módulo publicador (publisher) — cubierto por HU\_01.
- La persistencia de mensajes en base de datos — cubierta por HU\_05.
- Pruebas de rendimiento bajo alta concurrencia — planificadas para la fase de pruebas de sistema.

# 2. Arquitectura bajo prueba

El sistema implementa el patrón Publish/Subscribe sobre el protocolo MQTT utilizando un broker Mosquitto 2.x. Los componentes principales son:

- Broker MQTT (eclipse-mosquitto:2): intermediario de mensajería que enruta y distribuye mensajes entre publicadores y suscriptores.
- src/validador.py: módulo de lógica pura que valida el payload recibido (JSON íntegro con campos obligatorios). No tiene dependencias de red.
- src/subscriber.py: cliente MQTT (paho-mqtt 1.6.1) que gestiona la conexión, suscripción, recepción y confirmación (QoS 1 / PUBACK).
- src/main.py: punto de entrada que lee configuración de variables de entorno y arranca el loop bloqueante.

La orquestación se realiza con **Docker Compose**, que garantiza que el broker esté saludable (healthcheck) antes de iniciar el subscriber.

2.1 Niveles de prueba (ISO/IEC 12207:2017)

Conforme al proceso 6.4 de ISO/IEC 12207:2017, se definen dos niveles de prueba para este componente:

| Nivel                | Proceso ISO 12207                      | Módulo                               | Herramienta                           |
|----------------------|----------------------------------------|--------------------------------------|---------------------------------------|
| Capa 1 – Unitario    | § 6.4.3 – Software unit verification   | src/validador.py                     | pytest (sin broker – ejecución < 1s)  |
| Capa 2 – Integración | § 6.4.4 – Software integration testing | src/subscriber.py + broker Mosquitto | pytest + docker-compose (broker real) |

3. Criterios de calidad (ISO/IEC 25010:2023)

Cada nivel de prueba valida características del modelo de calidad definido en ISO/IEC 25010:2023:

| Característica | Subcaracterística                  | Nivel que la valida | Relación con HU_03                                                                 |
|----------------|------------------------------------|---------------------|------------------------------------------------------------------------------------|
| Fiabilidad     | Corrección funcional               | C1 + C2             | El mensaje se entrega íntegramente con todos sus campos (CA_01)                    |
| Fiabilidad     | Manejo de faltas                   | C1 + C2             | Errores tipados ante payload vacío, JSON inválido o campos faltantes (FA 4.1–4.3)  |
| Fiabilidad     | Disponibilidad                     | C2                  | El subscriber se reconecta automáticamente; mensajes inválidos no bloquean el loop |
| Fiabilidad     | Integridad                         | C2                  | Los datos del mensaje no se alteran en tránsito broker → subscriber                |
| Seguridad      | Resistencia a entradas malformadas | C1 + C2             | Payloads maliciosos o corruptos se rechazan antes de procesar                      |
| Compatibilidad | Interoperabilidad                  | C2                  | Wildcard MQTT '#' compatible con el broker Mosquitto 2.x                           |
| Rendimiento    | Capacidad                          | C1                  | Contenido de hasta 10.000 caracteres procesado sin error                           |
| Mantenibilidad | Consistencia                       | C1                  | Jerarquía de excepciones coherente (MensajeVacioError ⊂ MensajeInvalidoError)      |

4. Criterios de entrada y salida (ISO/IEC/IEEE 29119-3)

## 4.1 Criterios de entrada (Entry Criteria)

- El código fuente de `src/validador.py` y `src/subscriber.py` está disponible en el repositorio y ha pasado revisión de código.
- La imagen Docker `eclipse-mosquitto:2` es accesible desde el entorno de ejecución.
- El archivo `docker-compose.yml` valida sin errores (`docker-compose config`).
- Las dependencias de Python están instaladas según `requirements.txt`.
- Las variables de entorno `BROKER_HOST` y `BROKER_PORT` están definidas.

## 4.2 Criterios de salida (Exit Criteria)

- 100% de los 35 casos de Capa 1 pasan (`pytest tests/test_capa1_validador.py`).
- 100% de los 15 casos de Capa 2 pasan con broker real disponible.
- Cobertura de líneas  $\geq 90\%$  en `src/validador.py` según `pytest-cov`.
- Cobertura de líneas  $\geq 80\%$  en `src/subscriber.py`.
- Ningún test termina en estado ERROR (solo PASSED o XFAIL si aplica).
- Todos los flujos alternos del UC (FA 3.x, FA 4.x, FE 2.x) tienen al menos un caso de prueba en estado PASSED.

## 4.3 Criterios de suspensión y reanudación

- Suspensión: si el broker Mosquitto no está disponible, los tests de Capa 2 se suspenden. Los de Capa 1 continúan de forma independiente.
- Reanudación: una vez que el healthcheck del broker retorna OK, se reanuda la ejecución de Capa 2 desde el primer caso fallido.

## 5. Casos de prueba — Capa 1: Unitarios (validador.py)

Archivo: `tests/test_capa1_validador.py` | Total: **35 casos** | Proceso ISO 12207: § 6.4.3

Técnicas de diseño (ISO/IEC/IEEE 29119-4): partición de equivalencia, análisis de valor límite, pruebas paramétricas.

### V-01 · Mensajes válidos — Flujo básico paso 4

| ID      | Descripción                                | Entrada / Condición                                                         | Resultado esperado                             | Flujo UC    |
|---------|--------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------|-------------|
| V-01-01 | Payload completo retorna MensajeRecibido   | JSON con <i>id</i> , <i>categoria</i> , <i>contenido</i> , <i>timestamp</i> | Instancia de MensajeRecibido                   | Paso 4 – FB |
| V-01-02 | Campos mapeados correctamente al dataclass | JSON con <i>id</i> ='xyz-99', <i>categoria</i> ='cat/test'                  | <code>mensaje.id == 'xyz-99'</code>            | Paso 4 – FB |
| V-01-03 | Contenido puede ser objeto JSON anidado    | <i>contenido</i> :<br><code>{"sensor": "temp", "valor": 23.5}</code>        | <code>mensaje.contenido == dict anidado</code> | Paso 4 – FB |

| ID                 | Descripción                                  | Entrada / Condición                                         | Resultado esperado                 | Flujo UC    |
|--------------------|----------------------------------------------|-------------------------------------------------------------|------------------------------------|-------------|
| V-01-04            | Contenido puede ser lista JSON               | <i>contenido: [1,2,3]</i>                                   | mensaje.contenido == [1,2,3]       | Paso 4 – FB |
| V-01-05            | id numérico se convierte a string            | <i>id: 42 (int)</i>                                         | mensaje.id == '42'                 | Paso 4 – FB |
| V-01-06            | raw preserva dict original completo          | <i>JSON con campo_extra adicional</i>                       | 'campo_extra' in mensaje.raw       | Paso 4 – FB |
| V-01-07 al V-01-11 | Casos válidos paramétricos (5 combinaciones) | <i>Tabla: distintas categorías, contenidos y timestamps</i> | MensajeRecibido en todos los casos | Paso 4 – FB |

### V-02 · Payload vacío — Flujo alterno 4.1–4.3

| ID      | Descripción                                           | Entrada / Condición | Resultado esperado                          | Flujo UC   |
|---------|-------------------------------------------------------|---------------------|---------------------------------------------|------------|
| V-02-01 | Bytes vacíos lanzan MensajeVacioError                 | <i>b''</i>          | raise MensajeVacioError                     | FA 4.1–4.3 |
| V-02-02 | Solo espacios lanzan MensajeVacioError                | <i>b' '</i>         | raise MensajeVacioError                     | FA 4.1–4.3 |
| V-02-03 | Newline y tab lanzan MensajeVacioError                | <i>b'\n\t'</i>      | raise MensajeVacioError                     | FA 4.1–4.3 |
| V-02-04 | MensajeVacioError es subclase de MensajeInvalidoError | <i>b''</i>          | También capturado como MensajeInvalidoError | FA 4.1–4.3 |

### V-03 · JSON inválido — Flujo alterno 4.1–4.3

| ID                 | Descripción                                                             | Entrada / Condición                   | Resultado esperado                  | Flujo UC   |
|--------------------|-------------------------------------------------------------------------|---------------------------------------|-------------------------------------|------------|
| V-03-01            | Texto plano lanza MensajeNoEsJSONError                                  | <i>b'texto plano'</i>                 | raise MensajeNoEsJSONError          | FA 4.1–4.3 |
| V-03-02            | JSON malformado lanza MensajeNoEsJSONError                              | <i>b'{"id: sin-comillas}'</i>         | raise MensajeNoEsJSONError          | FA 4.1–4.3 |
| V-03-03            | JSON truncado lanza MensajeNoEsJSONError                                | <i>b'{"id": "msg"' (incompleto)</i>   | raise MensajeNoEsJSONError          | FA 4.1–4.3 |
| V-03-04            | Bytes no-UTF8 lanzan MensajeNoEsJSONError                               | <i>b'\xff\xfe invalido'</i>           | raise MensajeNoEsJSONError          | FA 4.1–4.3 |
| V-03-05            | Array JSON lanza MensajeInvalidoError                                   | <i>b'["id", "msg"]'</i>               | raise MensajeInvalidoError          | FA 4.1–4.3 |
| V-03-06            | Número JSON lanza MensajeInvalidoError                                  | <i>b'42'</i>                          | raise MensajeInvalidoError          | FA 4.1–4.3 |
| V-03-07 al V-03-11 | Payloads no-objeto paramétricos (null, true, false, string, doble JSON) | <i>5 variantes de payload no-dict</i> | raise MensajeInvalidoError en todos | FA 4.1–4.3 |

**V-04 · Campos requeridos faltantes**

| ID                                  | Descripción                                       | Entrada / Condición                                                            | Resultado esperado                 | Flujo UC    |
|-------------------------------------|---------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------|-------------|
| <b>V-04-01</b><br>al <b>V-04-04</b> | Cada campo requerido es obligatorio (paramétrico) | <i>JSON con id / categoria / contenido / timestamp omitido respectivamente</i> | raise CampoRequeridoError          | FA 4.1–4.3  |
| <b>V-04-05</b>                      | Objeto vacío {} lanza CampoRequeridoError         | <i>b'{'</i>                                                                    | raise CampoRequeridoError          | FA 4.1–4.3  |
| <b>V-04-06</b>                      | Mensaje de error menciona campos faltantes        | <i>JSON con solo id</i>                                                        | 'categoria' en str(exc_info.value) | FA 4.1–4.3  |
| <b>V-04-07</b>                      | Campos extra no generan error                     | <i>JSON completo + campo_extra</i>                                             | MensajeRecibido exitoso            | Paso 4 – FB |

**V-05 · Análisis de valor límite**

| ID             | Descripción                                  | Entrada / Condición           | Resultado esperado              | Flujo UC    |
|----------------|----------------------------------------------|-------------------------------|---------------------------------|-------------|
| <b>V-05-01</b> | Payload de 1 byte lanza MensajeNoEsJSONError | <i>b'{'</i>                   | raise MensajeNoEsJSONError      | FA 4.1      |
| <b>V-05-02</b> | Contenido de 10.000 caracteres es válido     | <i>contenido: 'x' * 10000</i> | len(mensaje.contenido) == 10000 | Paso 4 – FB |

## 6. Casos de prueba — Capa 2: Integración (subscriber.py + broker)

Archivo: `tests/test_capa2_integracion.py` | Total: 15 casos | Proceso ISO 12207: § 6.4.4

Requiere: broker Mosquitto disponible en `BROKER_HOST:BROKER_PORT` (ver `docker-compose.yml`).

### I-01 · Conexión al broker — Pasos 2–3 del flujo básico

| ID      | Descripción                                  | Entrada / Condición                                  | Resultado esperado                           | Flujo UC            |
|---------|----------------------------------------------|------------------------------------------------------|----------------------------------------------|---------------------|
| I-01-01 | Subscriber se conecta exitosamente al broker | <i>Broker disponible en BROKER_HOST:1883</i>         | <code>subscriber.conectado == True</code>    | Paso 2-3 – FB       |
| I-01-02 | Subscriber queda suscrito y recibe mensajes  | <i>Publicar PAYLOAD_VALIDO en axon/mensajes/test</i> | <code>len(mensajes_recibidos) &gt;= 1</code> | Paso 3 – FB · CA_01 |

### I-02 · Recepción íntegra — CA\_01

| ID      | Descripción                                  | Entrada / Condición                                   | Resultado esperado                                                                                       | Flujo UC            |
|---------|----------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------------|
| I-02-01 | Mensaje llega con todos sus campos intactos  | <i>PAYLOAD_VALIDO publicado con QoS 1</i>             | <code>msg.id</code> , <code>categoria</code> , <code>contenido</code> , <code>timestamp</code> coinciden | CA_01 · Paso 4 – FB |
| I-02-02 | No recibe mensajes fuera del topic suscrito  | <i>Publicar en 'otro/topic/diferente'</i>             | <code>mensajes_recibidos == []</code>                                                                    | CA_01 · Paso 2 – FB |
| I-02-03 | Wildcard '#' captura subtopics correctamente | <i>Publicar en axon/mensajes/alertas</i>              | <code>len(mensajes_recibidos) &gt;= 1</code>                                                             | CA_01 · Paso 3 – FB |
| I-02-04 | Múltiples mensajes se reciben en orden       | <i>3 publicaciones secuenciales con ids distintos</i> | Los 3 ids presentes en <code>mensajes_recibidos</code>                                                   | CA_01 · Paso 4 – FB |

### I-03 · Confirmación de recepción QoS 1 — CA\_02

| ID      | Descripción                                              | Entrada / Condición             | Resultado esperado                                    | Flujo UC            |
|---------|----------------------------------------------------------|---------------------------------|-------------------------------------------------------|---------------------|
| I-03-01 | QoS 1 garantiza entrega y confirmación PUBACK            | <i>Publicar con QoS 1</i>       | <code>len(mensajes_recibidos) == 1</code> sin errores | CA_02 · Paso 5 – FB |
| I-03-02 | Recepción exitosa no genera entradas en lista de errores | <i>PAYLOAD_VALIDO con QoS 1</i> | <code>subscriber.errores == []</code>                 | CA_02 · Paso 5 – FB |

### I-04 · Mensaje inválido — Flujo alternativo 4.1–4.3

| ID      | Descripción                                              | Entrada / Condición                           | Resultado esperado                                                          | Flujo UC   |
|---------|----------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------------------------|------------|
| I-04-01 | Payload no-JSON no se procesa ni se agrega a recibidos   | <i>b'esto no es json' publicado</i>           | <code>mensajes_recibidos == [] · errores contiene 'mensaje_invalido'</code> | FA 4.1–4.3 |
| I-04-02 | JSON sin campos requeridos se rechaza                    | <i>{"dato":"incompleto"}</i>                  | <code>mensajes_recibidos == []</code>                                       | FA 4.1–4.3 |
| I-04-03 | Payload vacío no se procesa                              | <i>b'' publicado</i>                          | <code>mensajes_recibidos == []</code>                                       | FA 4.3     |
| I-04-04 | Mensaje inválido no bloquea mensajes válidos posteriores | <i>Secuencia: b'no json' → PAYLOAD_VALIDO</i> | <code>len(mensajes_recibidos) == 1 (solo el válido)</code>                  | FA 4.1–4.3 |

I-05 · Broker no disponible — Flujo alterno 3.1–3.3

| ID      | Descripción                                                             | Entrada / Condición                                  | Resultado esperado                         | Flujo UC   |
|---------|-------------------------------------------------------------------------|------------------------------------------------------|--------------------------------------------|------------|
| I-05-01 | Conexión a host inexistente lanza <code>SubscriberConexionError</code>  | <i>host='host-que-no-existe.local', reintentos=1</i> | <code>raise SubscriberConexionError</code> | FA 3.1–3.3 |
| I-05-02 | Conexión a puerto incorrecto lanza <code>SubscriberConexionError</code> | <i>puerto=9999, reintentos=1</i>                     | <code>raise SubscriberConexionError</code> | FA 3.1–3.3 |

## 7. Matriz de trazabilidad

La siguiente tabla traza cada elemento del caso de uso GU.HU3.UC1 con los casos de prueba que lo cubren.

| Elemento UC | Descripción                              | Casos de prueba                        | Estado     |
|-------------|------------------------------------------|----------------------------------------|------------|
| Paso 4 – FB | Recibir mensaje íntegro                  | V-01-01 a V-01-11, I-02-01, I-02-04    | ✓ Cubierto |
| Paso 5 – FB | Confirmar recepción al broker (QoS 1)    | I-03-01, I-03-02                       | ✓ Cubierto |
| Paso 6 – FB | Continuar con proceso siguiente          | V-01-01 (callback on_mensaje_recibido) | ✓ Cubierto |
| CA_01       | Entrega íntegra y oportuna               | V-01-01 a V-01-11, I-02-01 a I-02-04   | ✓ Cubierto |
| CA_02       | Confirmar antes de continuar             | I-03-01, I-03-02                       | ✓ Cubierto |
| FA 3.1–3.3  | Broker no disponible → error registrado  | I-05-01, I-05-02                       | ✓ Cubierto |
| FA 4.1–4.3  | Mensaje inválido → no confirmar, detener | V-02, V-03, V-04, I-04-01 a I-04-04    | ✓ Cubierto |
| FE 2.1–2.2  | Suscriptor no conectado → rechazar       | I-01-01 (assert conectado)             | ✓ Cubierto |

## 8. Resumen de cobertura

| Nivel                | Archivo de test           | Módulo bajo prueba                   | Total casos |
|----------------------|---------------------------|--------------------------------------|-------------|
| Unitario (Capa 1)    | test_capa1_validador.py   | src/validador.py                     | 35          |
| Integración (Capa 2) | test_capa2_integracion.py | src/subscriber.py + broker Mosquitto | 15          |
| TOTAL                |                           |                                      | 50          |

Objetivo de cobertura de código: **≥ 90% de líneas en src/validador.py** y **≥ 80% en src/subscriber.py**, medido con pytest-cov ejecutando ambas capas de forma conjunta.

## 9. Riesgos y mitigaciones

| ID   | Riesgo                                       | Probabilidad | Impacto | Mitigación                                                                                             |
|------|----------------------------------------------|--------------|---------|--------------------------------------------------------------------------------------------------------|
| R-01 | Broker Mosquitto no disponible en entorno CI | Media        | Alto    | Usar healthcheck en docker-compose; reintentos exponenciales en subscriber (max_reintentos_conexion=5) |

| ID   | Riesgo                                                   | Probabilidad | Impacto | Mitigación                                                                                                             |
|------|----------------------------------------------------------|--------------|---------|------------------------------------------------------------------------------------------------------------------------|
| R-02 | Mensajes con formato incorrecto generan falsos negativos | Baja         | Medio   | Tests de Capa 1 cubren todos los casos de error tipados (MensajeVacioError, MensajeNoEsJSONError, CampoRequeridoError) |
| R-03 | Pérdida de mensajes con QoS 0 en red inestable           | Media        | Alto    | Configuración forzada a QoS 1 (PUBACK) – CA_02. Documentado en pytest.ini con timeout=15s                              |
| R-04 | Colisión de client_id en ejecución paralela de tests     | Baja         | Medio   | Fixture genera client_id único via time.time_ns() por cada test                                                        |
| R-05 | Regresión al modificar validador.py                      | Media        | Alto    | 35 tests unitarios de Capa 1 ejecutan sin broker en <1s; integrados en pipeline CI                                     |

## 10. Entorno de prueba y comandos de ejecución

### 10.1 Entorno

| Componente           | Versión / Configuración                                              |
|----------------------|----------------------------------------------------------------------|
| Broker MQTT          | <i>eclipse-mosquitto:2 · allow_anonymous=true · puerto 1883</i>      |
| Python               | <i>3.11-slim (imagen Docker)</i>                                     |
| paho-mqtt            | <i>1.6.1</i>                                                         |
| pytest               | <i>8.2.0 + pytest-cov 5.0.0 + pytest-timeout 2.3.1 (timeout=15s)</i> |
| Docker Compose       | <i>3.9 · healthcheck en broker · profiles: test, manual</i>          |
| Variables de entorno | <i>BROKER_HOST · BROKER_PORT · TOPIC · QOS · CLIENT_ID</i>           |

### 10.2 Comandos de ejecución

- Capa 1 únicamente (sin broker): `docker-compose run --rm tests pytest tests/test_capa1_validador.py -v`
- Capa 2 únicamente (requiere broker): `docker-compose up -d broker && docker-compose run --rm tests pytest tests/test_capa2_integracion.py -v`
- Suite completa con cobertura: `docker-compose run --rm tests`
- Publicar mensaje de prueba manual: `docker-compose run --rm publisher`

## 11. Aprobaciones

| Rol                    | Nombre           | Firma | Fecha |
|------------------------|------------------|-------|-------|
| Responsable de pruebas | Nicolas Saavedra |       |       |
| Revisor técnico        |                  |       |       |

| Rol                    | Nombre | Firma | Fecha |
|------------------------|--------|-------|-------|
| Aprobador del proyecto |        |       |       |